

How Linux Kernel Live Patching Works

Servers, systems or HPCs are meant to run on a 24x7x365 basis but due to critical security updates or OS updates, they have to be restarted to apply the changes. Linux kernel live patching takes care of this issue and avoids restarts.

Live patching starts with making a patch to change a specific kernel function. The patch can be created with a tool like *kpatch-build*. The result is a kernel module, which is then distributed. When this module is loaded, it ensures that processes using a particular system call are using its patched version, a process that is somewhat similar to a traffic diversion.

There are three kernel features that make the process of patching possible.

Kprobes: Kprobes or kernel probes is a feature used by developers to assess

the Linux kernel and perform debugging. Kprobes allows developers to break into kernel routines at many code addresses. This is called a breakpoint and allows the developer to take some action, such as to run a new set of instructions.

Ftrace: The next feature is called function tracer or Ftrace. This is a powerful framework to measure several aspects within the kernel, like events and interrupts. For example, it can measure the latency of specific functions like writing to disk.

Livepatch: Livepatch is the third component. It is also the latest addition to the kernel. With a custom Ftrace handler, it can redirect routines and jump to a patched set of instructions.

The life cycle of a live patch

Live patching involves four basic operations that define the life cycle of each live patch.

Registration: Each patch has first to be registered using *kpatch_register_patch()*. This makes the patch known to the live patch framework. After some preliminary computing and checking, the patch is added into the list of known patches. The addresses of the patched functions are found according to their names. The special relocations, mentioned in the section 'New functions', are applied. The relevant entries are created under */sys/kernel/livepatch/<name>*. The patch is rejected when any operation fails.

Enabling: Registered patches might be enabled either by calling *kpatch_enable_patch()* or by writing '1' to */sys/kernel/livepatch/<name>/enabled*. The system will start using the new implementation of the patched functions at this stage. In particular, if an original function is patched for the first time, a function-specific *struct klp_ops* is created and a universal Ftrace handler is registered.

Disabling: Enabled patches might get disabled either by calling *kpatch_disable_patch()* or by writing '0' to */sys/kernel/livepatch/<name>/enabled*. At this stage, either the code from the previously enabled patch or even the original code gets used.

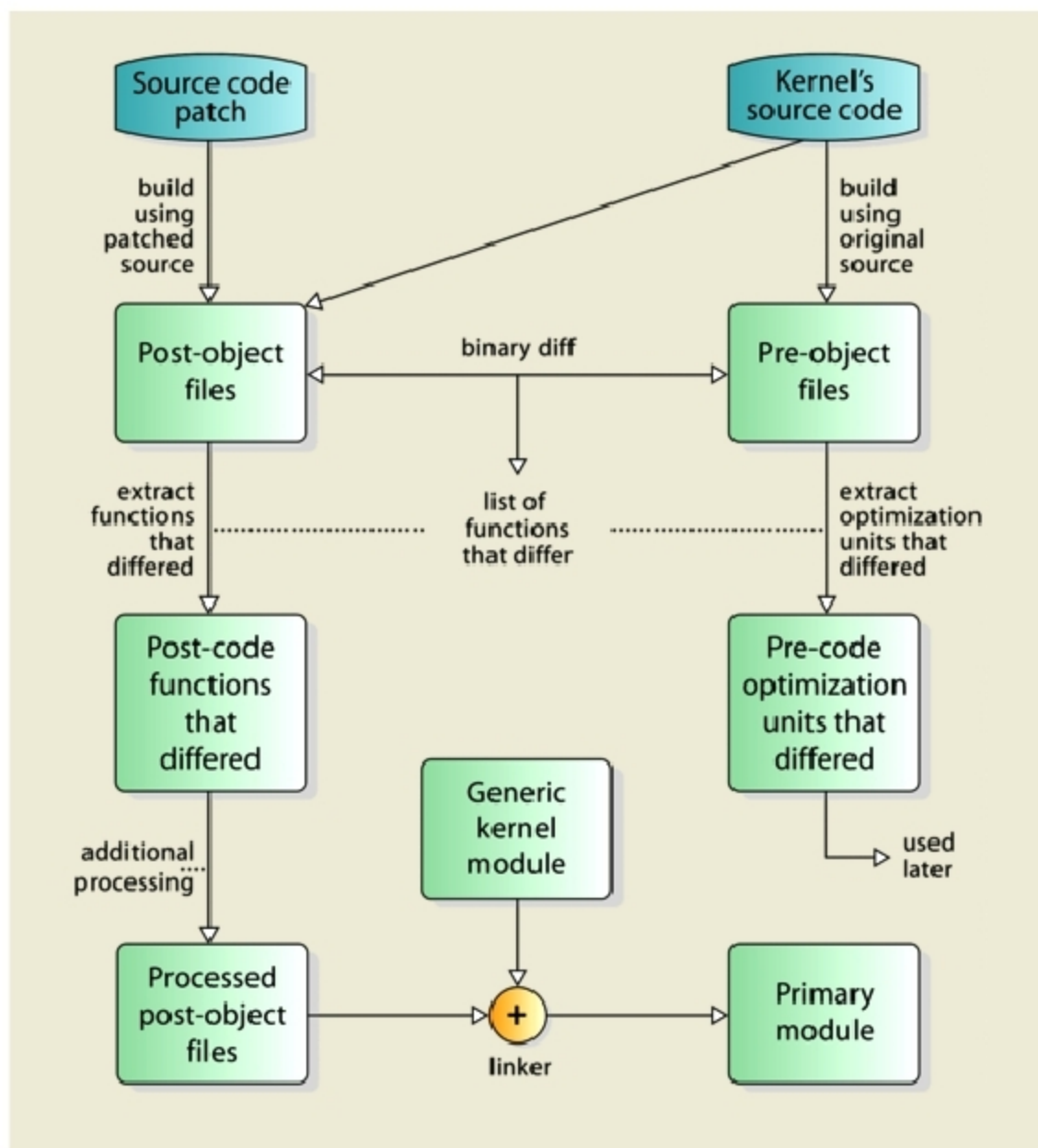


Figure 1: Generating replacement code based on a patch